
用户信息管理平台

安全漏洞挖掘与修复报告

http://192.168.43.129:5000

项目版本：V4.0 — SQL注入漏洞修复专题

报告日期：2026年7月8日

今日课程：Day2:SQL注入漏洞挖掘与修复

技术栈：Python Flask / SQLite / 参数化查询 / POC验证

安全评级：★★★★★ 25项防护全部通过

一、SQL注入漏洞专题

第2天课程内容为 SQL 注入漏洞的挖掘与修复。系统在上次迭代中新增了注册和搜索功能，但使用了 f-string 字符串拼接 SQL 语句，故意留下了注入漏洞。

4.1 漏洞位置

功能	路由	漏洞代码
用户注册	/register	f"INSERT INTO users VALUES ('{username}',...)"
用户搜索	/search	f"SELECT ... WHERE username LIKE '{keyword}%"

4.2 POC 验证（注入成功）

POC 1: UNION 注入

```
# 输入
keyword = ' UNION SELECT 1,'inj','inj@x.com','138'--

# 生成的 SQL
SELECT * FROM users WHERE username LIKE '%' UNION SELECT 1,'inj','inj@x.com','138'--%'

# 结果：搜索结果中出现 "inj" 用户 ✓
```

POC 2: OR 万能条件

```
# 输入
keyword = ' OR '1'='1

# 生成的 SQL
SELECT * FROM users WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'

# 结果：返回 users 表中全部用户数据 ✓
```

POC 3: 注册功能注入

```
# 输入
username = hacker', 'pass', 'h@x.com', '123')--

# 生成的 SQL
INSERT INTO users VALUES ('hacker', 'pass', 'h@x.com', '123')--,...)
```

结果：恶意数据写入数据库 

4.3 修复方案

将 f-string 拼接 SQL 改为参数化查询（Prepared Statement），根本性防止 SQL 注入：

```
// ❌ 修复前：f-string 拼接（存在注入）
sql = f"SELECT ... WHERE username LIKE '%{keyword}%"

// ✅ 修复后：参数化查询（安全）
sql = "SELECT ... WHERE username LIKE ?"
cursor.execute(sql, (like_pattern,))
```

4.3 修复前后代码对比

功能	修复前（f-string拼接）	修复后（参数化查询）
注册	f"INSERT INTO users VALUES ('{username}', '{password}', '{email}', '{phone}')"	sql = "INSERT INTO users VALUES (?, ?, ?, ?)" cursor.execute(sql, (username, password, email, phone))
搜索	f"SELECT ... WHERE username LIKE '%{keyword}%"	sql = "SELECT ... WHERE username LIKE ?" cursor.execute(sql, (like_pattern,))

4.4 Burp Suite 测试方法

使用 Burp Suite 抓包后修改 keyword 参数测试注入：

1. 登录后拦截 GET /search?keyword=admin 请求
2. 发送到 Repeater
3. 修改 keyword 参数值测试注入：
admin' OR '1'='1 → 返回全部用户
' UNION SELECT 1,2,3,4-- → 返回自定义数据

4.5 修复后验证

测试	修复前	修复后
POC 1 UNION 注入	返回 "inj" 数据	[OK] 注入无效，搜索结果为0
POC 2 OR 万能条件	返回全部用户	[OK] 注入无效，正常搜索
POC 3 注册注入	SQL代码被执行	[OK] 注入内容成为普通用户名

二、总结

本次SQL注入漏洞修复专题，针对注册和搜索功能中的 f-string 拼接SQL漏洞进行了全面修复：

课程内容	受影响功能	漏洞类型	修复措施
SQL注入	注册 / 搜索	f-string拼接SQL	参数化查询(Prepared Statement)

核心原理：参数化查询将 SQL 语句与用户输入的数据分离，数据库先编译 SQL 结构（SELECT、INSERT 等），再将用户输入作为"纯数据"填入。即使用户输入包含恶意 SQL 代码，也不会被数据库执行，从根源消除注入风险。

— 报告完 —

报告日期: 2026-07-08 | 课程: SQL注入漏洞修复 | 安全标准: OWASP